TSwap Audit Report

Version 1.0

Reynald_Olinga

March 9, 2026

TSwap Audit Report

Reynald Olinga

March 9, 2026

Prepared by: Reynald Olinga

Lead Auditors: Reynald Olinga

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
  - HIGH ISSUES
  - [H-1]: Reentrancy: State change after external call
    * [H-2] Incorrect fee calculation in `TswapPool::getInputAmountBasedOnOutput` causes to take too many tokens from users resulting in lost fees
    * [H-3] lack of slippage protection in `TSwapPool::swapExactOutput` causes users to potentially receive way fewer tokens:
    * [H-4] `TSwapPool::sellPoolTokens` mismatches input and output tokens causing users receive the incorrect amount of tokens

    * [H-5] In TSwapPool::_swap the extra tokens given to users after every swapCount breaks the protocol invariant of x * y = k
  - MEDIUM
    * [M-1] `TswapPool::deposit` is missing deadline check causing transactions to complete even after the deadline
  - LOW ISSUES
    * [L-1] `TswapPool::LiquidityAdded` event has parameters out of order
  - [L-2] Default value returned by `TSwapPool::swapExactInput` results in incorrect return value given
  - INFORMATIONALS
    * [I-1] `PoolFactory::PoolFactory__PoolDoesNotExist` is not used and should be removed
    * [I-2] Lacking zero address checks
    * [I-3] `PoolFactory::createPool` should use `.symbol()` instead of `.name`

Protocol Summary

The protocol starts as simply a `PoolFactory` contract. This contract is used to create new "pools" of tokens. It helps make sure every pool token uses the correct logic. But all the magic is in each `TSwapPool` contract.

You can think of each `TSwapPool` contract as it's own exchange between exactly 2 assets. Any ERC20 and the WETH token. These pools allow users to permissionlessly swap between an ERC20 that has a pool and WETH. Once enough pools are created, users can easily "hop" between supported ERC20s.

For example: 1. User A has 10 USDC 2. They want to use it to buy DAI 3. They `swap` their 10 USDC -> WETH in the USDC/WETH pool 4. Then they `swap` their WETH -> DAI in the DAI/WETH pool

Every pool is a pair of `TOKEN X` & `WETH`.

There are 2 functions users can call to swap tokens in the pool. - `swapExactInput` - `swapExactOutput`

We will talk about what those do in a little.

Disclaimer

Reynald OLINGA makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is

not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Scope

```
./src/
#-- PoolFactory.sol
#-- TSwapPool.sol
```

Roles

Executive Summary

This security assessment was conducted on the TSwap Pool which is meant to be a permissionless way for users to swap assets between each other at a fair price. You can think of T-Swap as a decentralized asset/token exchange (DEX). T-Swap is known as an Automated Market Maker (AMM) because it doesn't use a normal "order book" style exchange, instead it uses "Pools" of an asset. It is similar to Uniswap. To understand Uniswap, please watch this video: Uniswap Explained

The objective of this audit was to evaluate the protocol's security, correctness, and robustness, with a specific focus on the x * y = k invariant

The findings outlined in this report are accompanied by practical remediation recommendations aligned with industry best practices. Implementing these recommendations will significantly improve the protocol's safety, reliability, and trustworthiness prior to deployment in a production environment.

Issues found

Severity	Number of issues found
High	4
Medium	2
Low	2
Info	9
Total	17

HIGH ISSUES

[H-1]: Reentrancy: State change after external call

Changing state after an external call can lead to re-entrancy attacks.Use the checks-effects-interactions pattern to avoid this issue.

2 Found Instances

- Found in src/PoolFactory.sol Line: 51

  State is changed at: `s_pools[tokenAddress] = address(tPool), s_tokens[address(tPool)] = tokenAddresssolidity string memory liquidityTokenName = string.concat(`

- Found in src/PoolFactory.sol Line: 55

  State is changed at: `s_pools[tokenAddress] = address(tPool), s_tokens[address(tPool)] = tokenAddresssolidity string memory liquidityTokenSymbol = string.concat(`

[H-2] Incorrect fee calculation in `TswapPool::getInputAmountBasedOnOutput` causes to take too many tokens from users resulting in lost fees

Description:

The `getInputAmountBasedOnOutput` function is intended to calculate the amount of token a user should depoisit given on amount of token of output tokens. However the function currently miscalculate the resultingn amount. When calculating the fee, it scales the amount by 10000 instead of 1000

Impact:

Protocol takes more fee than expected from user

Proof of Concept:

Recommended Mitigation:

```
function getInputAmountBasedOnOutput(
        uint256 outputAmount,
        uint256 inputReserves,
        uint256 outputReserves
    )
        public
        pure
        revertIfZero(outputAmount)
        revertIfZero(outputReserves)
        returns (uint256 inputAmount)
    {
        return
-            ((inputReserves * outputAmount) * 10000) /
+            ((inputReserves * outputAmount) * 1000) /
            ((outputReserves - outputAmount) * 997);
    }
```

[H-3] lack of slippage protection in `TSwapPool::swapExactOutput` causes users to potentially receive way fewer tokens:

Description:

The `swapExactOutput` function not include any sort of slippage protection. This function is similar to what is done in `TSwapPool::swapExactInput`,where the function specifies a `minOutputAmount` the `swapExactOutput` function shuld specify a `maxInputAmount`

Impact:

If market conditions change before the transaction processes the user could get a much worse swap

Proof of Concept:

1. The price of 1 WETH right now is 1000 USDC
2. user inputs a `swapExactOutput` looking for 1 WETH
    1. INPUTtOKEN = USDC
    2. outputToken = WETH
    3. outputAmount = 1
    4. deadline = whatever
3. The function does not offer a `maxInputAmount`
4. As the transaction is pending in the mempool, the market changes ! And the price moves HUGE -> 1 WETH is now 10000 USDC. 10x more than the user expected
5. The transaction completes but the user sent the protocol 10000 USDC instead of the expected 1000 USDC

Recommended Mitigation:

We should include a `maxInputAmount` so the user only has to spend up to a specific amount and can predict how much they will spend on the protocol

```
function swapExactOutput(
    IERC20 inputToken,
    IERC20 outputToken,
+    uint256 outputAmount,
.
.
.
inputAmount = getInputAmountBasedOnOutput(outputAmount, inputReserves, outputReserves);
+    if(inputAmount > maxInputAmount) {
+        revert();
+    }

_swap(inputToken, inputAmount, outputToken, outputAmount);
```

[H-4] `TSwapPool::sellPoolTokens` mismatches input and output tokens causing users receive the incorrect amount of tokens

Description:

The `sellPoolTokens` function is intended to allow users to easily sell pool tokens and receive WETH in exchange. Users indicate how many pool tokens they are willing to sell in the `poolTokenAmount` parameter. However, the function currently miscalculate the swapped amount

This is due to the fact that the `swapExatOutput` function is called whereas the `swapExactInput` function is the one that should be called. Because users specify the exact amount of input Tokens not output.

Impact:

Users will swap the wrong amount of tokens, which is a severe disruption of protocol functionality.

Proof of Concept:

```
function sellPoolTokens(
        uint256 poolTokenAmount
        uint256 minWethToReceive
    ) external returns (uint256 wethAmount) {
-       return swapExactOutput(i_poolToken, i_wethToken,
    poolTokenAmount, uint64(block.timestamp) );
+       return swapExactInput(i_poolToken, poolTokenAmount, i_wethToken
    , minWethToReceive uint64(block.timestamp));}
```

Recommended Mitigation:

Consider changing the implementation to use `swapExactInput` instead of `swapExactOutput`. Note that this would also require changing the `sellPoolTokens` function to accept a new parameter (ie minWethToReceive to be passed to swapExactInput)

[H-5] In TSwapPool::_swap the extra tokens given to users after every swapCount breaks the protocol invariant of x * y = k

Description:

The protocol follows a strict invariant of x * y = k. Where:

- x: The balance of the pool token
- y: The balance of WETH
- k: The constant product of the two balances

This means, that whenever the balances change in the protocol, the ratio between the two amounts should remain constant, hence the k. However, this is broken due to the extra incentive in the _swap function. Meaning that over time the protocol funds will be drained.

The follow block of code is responsible for the issue.

```
        swap_count++;
        if (swap_count >= SWAP_COUNT_MAX) {
        swap_count = 0;
```

```
        outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);
        }
```

Impact:

A user could maliciously drain the protocol of funds by doing a lot of swaps and collecting the extra incentive given out by the protocol. Most simply put, the protocol's core invariant is broken.

Proof of Concept:

1. A user swaps 10 times, and collects the extra incentive of 1_000_000_000_000_000_000 tokens.
2. That user continues to swap until all the protocol funds are drained

Proof of Code

Place the following into `TSwapPool.t.sol`

```


    function testInvariantBroken() public {
        vm.startPrank(liquidityProvider);
        weth.approve(address(pool), 100e18);
        poolToken.approve(address(pool), 100e18);
        pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
        vm.stopPrank();

         uint256 outputWeth = 1e17;

        vm.startPrank(user);
        poolToken.approve(address(pool), type(uint256).max);
        poolToken.mint(user, 100e18);
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));
        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.timestamp));

```

```
        int256 startingX = int256(weth.balanceOf(address(pool)));
        int256 expectedDeltaX = int256(outputWeth) * -1;

        pool.swapExactOutput(poolToken, weth, outputWeth, uint64(block.
            timestamp));
        vm.stopPrank();

        int256 endingX = int256(weth.balanceOf(address(pool)));
        int256 actualDeltaX = int256(endingX) - int256(startingX);

        assertEq(actualDeltaX, expectedDeltaX);
    }
```

Recommended Mitigation:

Remove the extra incentive mechanism. If you want to keep this in, we should account for the change in the x * y = k protocol invariant. Or, we should set aside tokens in the same way we do with fees.

```
-        swap_count++;
-        // Fee-on-transfer
-        if (swap_count >= SWAP_COUNT_MAX) {
-            swap_count = 0;
-            outputToken.safeTransfer(msg.sender, 1
    _000_000_000_000_000_000);
-        }
```

MEDIUM

[M-1] `TswapPool::deposit` is missing deadline check causing transactions to complete even after the deadline

Description:

The `deposit` function accept a deadline parameter whih according to the documentation is "/// @param deadline The deadline for the transaction to be completed by". However this parameter is never used. As consequence operrations that add liquidity to the pool might be executed at unexpected times in th market conditions where the deposit rate is unfavorable

Impact:

Transactions can be send when the conditions of the market are unfavorable to deposit, even when adding a deadline parameter.

Proof of Concept:

The `deadline` parameter is unused

Recommended Mitigation:

Consider making the following changes to the function

```
function deposit(
        uint256 wethToDeposit,
        uint256 minimumLiquidityTokensToMint, // LP tokens -> If it is empty, we can pick 100% (100% == 17 tokens )
        uint256 maximumPoolTokensToDeposit,
        uint64 deadline
    )
        external
+      revertIfDeadlinePassed(deadline)
        revertIfZero(wethToDeposit)
        returns (uint256 liquidityTokensToMint)
```

LOW ISSUES

[L-1] `TswapPool::LiquidityAdded` event has parameters out of order

Description:

When the `LiquidityAdded` event is emitted in the `TswapPool::_addLiquidityMintAndTransfer` function, it logs values in an incorrect order. The `poolTokensToDeposit` value should go in the third parameter position, whereas the `wethToDeposit` value should go

Impact:

Event emission is incorrect leading to off-chain functions potentially malfunctioning

Proof of Concept:

Recommended Mitigation:

```
-    emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit);
+   emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit);
```

[L-2] Default value returned by `TSwapPool::swapExactInput` results in incorrect return value given

Description

The swapExactInput function is expected to return the actual amount of tokens bought by the caller. However, while it declares the named return value output it is never assigned a value, nor uses an explicit return statement.

Impact

The return value will always be 0, giving incorrect information to the caller.

Recommended Mitigation:

```
{
   uint256 inputReserves = inputToken.balanceOf(address(this));
   uint256 outputReserves = outputToken.balanceOf(address(this));
-        uint256 outputAmount = getOutputAmountBasedOnInput(inputAmount, inputReserves, outputReserves);
+        output = getOutputAmountBasedOnInput(inputAmount, inputReserves, outputReserves);
-        if (output < minOutputAmount) {
-            revert TSwapPool__OutputTooLow(outputAmount, minOutputAmount);
+        if (output < minOutputAmount) {
+            revert TSwapPool__OutputTooLow(outputAmount, minOutputAmount);
}
-        _swap(inputToken, inputAmount, outputToken, outputAmount);
+        _swap(inputToken, inputAmount, outputToken, output);
}
}
```

INFORMATIONALS

[I-1] `PoolFactory::PoolFactory__PoolDoesNotExist` is not used and should be removed

```
- error PoolFactory__PoolDoesNotExist(address tokenAddress)
```

[I-2] Lacking zero address checks

```
        constructor(address wethToken) {
+           if(wethToken == address(0)) {
+               revert();
+           }
        i_wethToken = wethToken;
}
```

[I-3] `PoolFactory::createPool` should use `.symbol()` instead of `.name`

```
-       string memory liquidityTokenName = string.concat("T-Swap ", IERC20(tokenAddress).name());

+       string memory liquidityTokenName = string.concat("T-Swap ", IERC20(tokenAddress).symbol());
```